



CSEE 4290: Computer Architecture

Top Level SCC – Spring 2025 – Group 5

[https://github.com/Herring-UGACSEE-4290-F25/Group-5/tree/main/Proj2\(SCC\)/EXEComplete](https://github.com/Herring-UGACSEE-4290-F25/Group-5/tree/main/Proj2(SCC)/EXEComplete)

Completed

Instruction Fetch

Decode

Execute

Register File

Fix Unconditional Branch Bubble

Implement uCode MUL/S

Create Testbench

To-Do

Write Unique Test Case

Run All Class Test Cases

Group Responsibilities

Test Bench: Jordan

Instruction Fetch: Ricky

Instruction Decode: Matthew

Execute: Nathaniel

uCode MUL/S: Everybody

Schedule

ID/IF: 10-03

Execute 3 Instructions/Branch Imm: 10-10

Execute Complete: 10-20

Testcases: 10-24

SCC Complete: 10-29

Testcase Plan

- Check writability to R15
- Large multiply with overflow to small value
- Check ORS and XORS
- Check BR, SET, and NOP
- Doublecheck if MOVT and MOVF works in regular program as well

ISA Map

- MOV: Bubble Sort
- MOVT: Multiply (uCode)
- MOVF: Multiply (uCode)
- LOAD: Bubble Sort
- STOR: Bubble Sort
- MUL: Multiply uCode Test Case
- MULS: Multiply uCode Test Case

- ADD: Bubble Sort
- ADDS: Multiply (uCode)
- SUB: Bubble Sort
- SUBS: Bubble Sort
- AND: Multiply
- ANDS: Multiply
- OR
- ORS
- XOR
- XORS
- NOT: Multiply
- B: Bubble Sort
- B.cond: Bubble Sort
- BR
- HALT: Bubble Sort
- LSL: Multiply
- LSR: Multiply
- CLR: Bubble Sort
- SET
- NOP

Testbench

Our testbench compares the data memory output of our SCC against the output of the emulator to see if they match.

```
`timescale 1ns/1ns
```

```
module scc_tb;
```

```
  localparam MEM_BYTES = 1<<16;
```

```
  reg clk_tb = 0;
```

```
  reg rst_tb = 1;
```

```
  reg clk_en_tb = 1;
```

```

wire halt_f_tb;

wire [1:0] err_bits_tb;

wire [31:0] instruction_memory_v_tb;

wire [31:0] data_memory_in_v_tb;


scc_f25_top dut (

    .clk(clk_tb),

    .rst(rst_tb),

    .clk_en(clk_en_tb),

    .halt_f(halt_f_tb),

    .err_bits(err_bits_tb),

    .instruction_memory_v(instruction_memory_v_tb),

    .data_memory_in_v(data_memory_in_v_tb)

);


always #5 clk_tb = ~clk_tb;


// release reset after 3 clock cycles
initial begin

    repeat (3) @(posedge clk_tb);

    rst_tb = 1'bo;

end

```

```
reg [7:0] dut_mem [0:MEM_BYTES-1];
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0, scc_tb);
```

```
end
```

```
// compare memory dump to CSV file
```

```
initial begin : COMPARE
```

```
    integer fd;
```

```
    integer matched;
```

```
    integer line_num;
```

```
    reg [255:0] line;
```

```
    reg [31:0] addr, value;
```

```
    reg [31:0] got_word;
```

```
// wait until HALT from the SCC
```

```
wait(halt_f_tb);
```

```
$display("Apoll has Landed!");
```

```
@(posedge clk_tb); // wait a clock after HALT to let $writememh finish in  
instruction_and_data
```

```
// load SCC memory dump file
```

```
$readmemh("memory_dump.mem", dut_mem);
```

```

// open CSV file
fd = $fopen("dataoutput.csv", "r");
if (fd == o) begin
    $display("ERROR: could not open dataoutput.csv");
    $finish;
end

matched = $fgets(line, fd); // read and ignore top line in dataoutput file

line_num = 1;
while (!$feof(fd)) begin
    line_num = line_num + 1;
    if ($fgets(line, fd)) begin

        // tolerate optional spaces and optional ox prefixes
        matched = $sscanf(line, "ox%h , ox%h", addr, value);
        if (matched != 2) matched = $sscanf(line, "ox%h,ox%h", addr, value);
        if (matched != 2) matched = $sscanf(line, "%h , %h", addr, value);
        if (matched != 2) matched = $sscanf(line, "%h,%h", addr, value);
    end
end

```

```

if (matched == 2) begin
    // bounds check
    if ((addr+3) >= MEM_BYTES) begin
        $display("FAIL @ line %0d: addr 0x%08h out of range", line_num, addr);
        $finish;
    end

    // assemble word: MSB at lowest address (matches your mem read logic)
    got_word = {dut_mem[addr+0], dut_mem[addr+1], dut_mem[addr+2],
dut_mem[addr+3]};

    if (got_word != value) begin
        $display("FAIL @ line %0d: addr=0x%08h got=0x%08h exp=0x%08h",
            line_num, addr, got_word, value);
        $finish;
    end
end

end

end

$display("PASS: all CSV memory values match DUT dump.");
$fclose(fd);
$finish;
end

```

endmodule

Testcase

The test case used for this checkpoint was our multiply instruction but ran without microcode or setting the flags back to their original state. The algorithm is explained in more detail in the Multiply section.

START:

ADD R0, R14, #10000; //Operand 1

ADD R1, R14, #5; //Operand 2

OPER1:

SUBS R0, R0, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER1NEG;

OPER2:

SUBS R1, R1, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER2NEG;

OPERATION:

ANDS R3, R1, #1; //LSB of multiplier

B.EQ OPERATIONNEXT;

ADD R2, R2, R0;

OPERATIONNEXT:

LSL R0, R0, #1;

LSR R1, R1, #1;

SUBS R1, R1, #0x0; //See if multiplier is 0

B.EQ OUTPUT;

B OPERATION;

OPER1NEG:

NOT R0, R0; //2's complement

ADD R0, R0, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPER2;

OPER2NEG:

NOT R1, R1; //2's complement

ADD R1, R1, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPERATION;

OUTPUT:

ANDS R7, R5, #1;

HALT

Bubble Sort

ORG #0x400;

FCB #0x5;

FCB #0x3;

FCB #0x7;

FCB #0x1;

FCB #0x4;

```
FCB    #0x9;
FCB    #0x6;
FCB    #0x8;
FCB    #0x2;
FCB    #0xA;
ORG    #0x0;
```

setup:

```
MOV    R0, #0x400;
MOV    R4, #0x400;  R4 placeholder for array
MOV    R5, #0x9;   length of array
MOV    R7, #0x0;   loop count value
```

mainloop:

```
CMP    R5, #0x0;
B.eq   exit;       exit if length is zero
LOAD   R1, R4;     initial pointer to array
```

iter:

```
CMP    R5, R7;
B.eq   iterexit;   exit pass if count = length
B      compare;
```

compreturn:

```
ADD    R4, R4, #0x4;  increment array pointer by 4
ADD    R7, R7, #0x1;  increment count
LOAD   R1, R4;       set value from pointer
```

B iter;

iterexit:

SUB R4, R4, R4;

MOV R4, #0x400; reset address to current first value

SUB R5, R5, #0x1; subtract one from length

CLR R7; reset count to 0

B mainloop;

compare:

LOAD R2, R4, #0x4; load second value to compare

CMP R2, R1; compare two values

B.mi swap; swap if $b - a < 0$

B compretreturn;

swap:

STOR R1, R4, #0x4; #store value one in address two

STOR R2, R4; #store value two in address one

B compretreturn;

exit:

HALT;

Multiply Algorithm

- First checks if the multiplicand is 0 or a negative with a SUBS #0. If the value is 0, then it branches to the end. If the value is negative, then it branches to make a 2's complement of the multiplicand and adds 1 to a counter that keeps track of how many negative operators the MULT had (R5). If the value is neither a 0 nor negative, then it moves on to check the multiplier with the exact same process.

- Once both operators are positive, the algorithm then checks the LSB of the multiplier to see if it is a 1 by using the ANDS instruction. If it is, then the value of the multiplicand is added to the product register (R2).
- The multiplicand is then shifted left by 1 and the multiplier right by 1, and the process above is repeated until the multiplier is equal to 0.
- The algorithm then used another ANDS on R5 to check how many negative operands the MULT had and determines whether the output should be negative or not. If the output should be negative, then it converts R2 to its 2's complement and goes on to set the proper flags. If the output is positive, then it branches to set the flags.
- The flags are always returned to their original state (Using a series of ANDS to check against the flags in R4 which was set by MOVF). The flags will then be newly set by the final value of the multiply depending on if it was a MUL or MULS instruction.

START:

ADD R0, R14, R0; //Operand 1

ADD R1, R14, #0; //Operand 2

MOVF R4; //Original Flag

OPER1:

SUBS R0, R0, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER1NEG;

OPER2:

SUBS R1, R1, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER2NEG;

OPERATION:

ANDS R3, R1, #1; //LSB of multiplier

B.EQ OPERATIONNEXT;

ADD R2, R2, R0;

OPERATIONNEXT:

LSL R0, R0, #1;

LSR R1, R1, #1;

SUBS R1, R1, #0x0; //See if multiplier is 0

B.EQ OUTPUT;

B OPERATION;

OPER1NEG:

NOT R0, R0; //2's complement

ADD R0, R0, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPER2;

OPER2NEG:

NOT R1, R1; //2's complement

ADD R1, R1, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPERATION;

OUTPUT:

ANDS R7, R5, #1;

B.EQ FLAGSHLL;

NEGATIVEOUT:

NOT R2, R2; //2's complement

ADD R2, R2, #1;

FLAGSHELL:

ANDS R6,R4,#8; //Masking for the N flag and checking if it's on

B.EQ NEG;

POS:

ANDS R6,R4,#4; //Masking for the Z flag

B.NE ZERO;

ANDS R6,R4,#2; //Masking for the C flag

B.NE CARRY;

ANDS R6,R4,#1; //Masking for the V flag

B.NE OVER;

MOV32 R0,#1;

MOV32 R1,#0;

ADDS R6,R0,R1; //Setting no flag

B FINALDONE;

OVER:

MOV32 R0,#0xFFFFFFFF;

MOV32 R1,#0x80000000;

SUBS R6,R0,R1; //Setting V flag

B FINALDONE;

CARRY:

ANDS R6,R4,#1; //Masking for CV flag

B.NE CARRYV;

MOV32 R0,#1;

MOV32 R1,#0;

SUBS R6,R0,R1; //Setting C flag

CARRYV:

MOV32 R0,#0xFFFFFFFF;

MOV32 R1,#0x80000000;

ADDS R6,R0,R1; //Setting CV flags

ZERO:

ANDS R6,R4,#2; //Masking for ZC flags

B.NE ZEROC;

MOV32 R0,#0;

MOV32 R1,#0;

ADDS R6,R0,R1; //Setting Z flag

B FINALDONE;

ZEROC:

ANDS R6,R4,#1; //Masking for ZCV flags

B.NE ZEROVC;

MOV32 R0,#1;

MOV32 R1,#0xFFFFFFFF;

ADDS R6,R0,R1; //Setting ZC flags

B FINALDONE;

ZEROVC:

MOV32 R0,#0x80000000;

MOV32 R1,#0x80000000;

ADDS R6,R0,R1; //Setting ZCV flags

B FINALDONE;

NEG:

ANDS R6,R4,#2; //Masking for NC flags

B.NE NEGC;

ANDS R6,R4,#1; //Masking for NV flags

B.NE NEGV;

MOV32 R0,#0xFFFFFFFF;

MOV32 R1,#1;

ADDS R6,R0,R1; //Setting N flag

B FINALDONE;

NEGV:

MOV32 R0,#1;

MOV32 R1,#0x7FFFFFFF;

ADDS R6,R0,R1; //Setting NV flags

B FINALDONE;

NEGC:

MOV32 R0,#0xFFFFFFFF;

MOV32 R1,#1;

ADDS R6,R0,R1; //Setting NC flags

B FINALDONE;

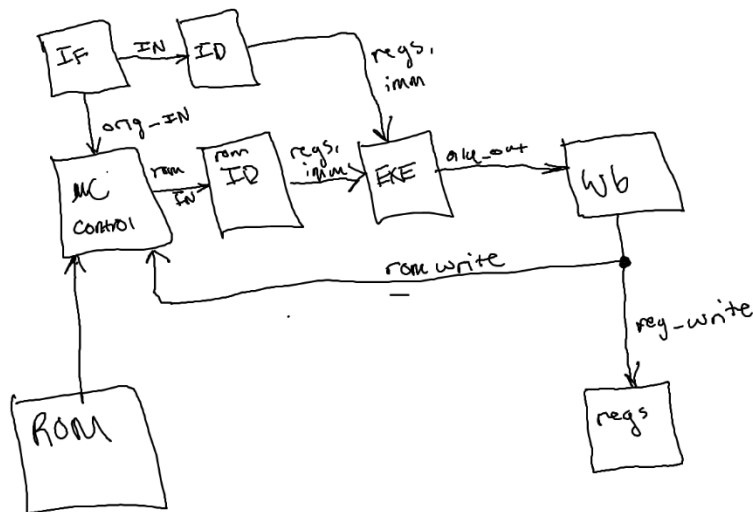
FINALDONE:

ADD R2, R2, #0;

HALT;

uCode Handler

Our uCode handler has its own IF stage and set of registers. Once a multiply instruction is detected, the main program's IF stage will halt and the uCode will take over. The uCode's IF stage acts exactly like the main program's except it will inject/change some ROM instructions to allow the uCode to import values from the main registers into the ROM registers, to export the output value from the ROM register to the main destination register (And set flags if MUL/I S was used), and to set the immediate values as an operand if a MULI/S was used. Both the uCode handler and the main program share the same ID, EXE, and WB stages. The uCode knows the multiply is over once it sees a HALT instruction in the ROM.



AFTER MC MUL:

- Send `ADD Rd, R2, #0;`
- Send instructions for flags

~~have wb in MC so wb~~